

说明

ARGB 外设可直接生成 WS2812、SK6812 等多种灯控协议时序，提供高时序精准度、低内存开销、低 CPU 占用的解决方案，适用于大规模灯光系统、电竞灯效、汽车灯效和音响灯效等场景。

适用范围

适用范围	系列
互连型 MCU	CH32V467、CH32V407、CH32X315
蓝牙 SoC	CH587

目录

说明	i
目录	ii
表格索引	ii
第 1 章 ARGB 优势	1
1.1 WS2812 主流控制方案对比	1
1.2 ARGB 的主要优势	1
第 2 章 使用方法	2
2.1 软件开发流程	2
2.2 典型应用场景	4
历史版本	5
声明	6

表格索引

表 1-1 WS2812 主流控制方案对比	1
-----------------------------	---

第1章 ARGB 优势

1.1 WS2812 控制方案对比

表 1-1 WS2812 控制方案对比

对比项	控制方案			
	ARGB	PIOC	TIM (PWM+DMA)	SPI+DMA
内存开销	低	低	高	高
	直接发送原始 RGB 数据，数据不膨胀	直接发送原始 RGB 数据，数据不膨胀	每个颜色 bit 转换为 1 或 2 字节数据，数据膨胀 8 倍或 16 倍	每个颜色 bit 转换为 1 或 2 字节数据，数据膨胀 8 倍或 16 倍
CPU 占用率	低	不占用	中等	中等
	仅处理中断	内核仅下发任务 PIOC 独立运行	数据预处理+中断处理	数据预处理+中断处理
时序精准度	高	高	较高	中等
	专用外设，频率、0/1 码高电平长度、复位码长度均可配置	汇编控制，时序精确到单条指令周期	取决于具体设计，可实现较高精度	受时钟频率和分频系数限制，可能无法完美满足时序要求
代码复杂度	低	中等	中等	中等
	仅配置 ARGB 外设和 DMA	提供示例代码，调用简单，上手容易，二次开发需了解汇编	需配置定时器和 DMA 协议由软件控制，数据需预处理	需配置 SPI 和 DMA 协议由软件控制，数据需预处理
灵活性	高	极高	中等	中等
	可配置参数多，支持 ARGB LED 型号多，兼容性好	支持协议自定义，时序精度高，灵活性好，兼容性好	受协议代码影响	受协议代码影响

1.2 ARGB 专用外设的主要优势

使用方便，开发快速直观：相较于 SPI+DMA 或 TIM+DMA 方案，ARGB 专用外设无需在每次刷新前对数据做预处理，数据直接支持 GRB 格式，与 WS2812 原生格式一致。PIOC 的协议代码提供参考示例，可直接调用。

没有“数据膨胀”：SPI+DMA 或 TIM+DMA 方案伴随成倍于原始数据的内存消耗。对于长灯带，例如 1024 个灯珠，使用常规的 SPI+DMA 或 TIM+DMA 方式，内存开销会从 3KB 增加到 24KB，造成内存资源浪费。ARGB 专用外设和 PIOC 不会造成数据膨胀。

高兼容性：ARGB 外设允许用户自由配置 0 码/1 码的高电平宽度(T0H、T1H)，0 码/1 码的总周期(DAT_ARR)，复位信号宽度(RST_ARR)。通过调整上述参数，可兼容 WS2812、SK6812(RGBW)、TM1814、UCS1903、GS8208 等不同厂商、不同时序要求的 ARGB 器件。

极低 CPU 占用、解放算力：在控制长灯带（如 1000+颗灯珠）时，CPU 占用率接近 0%。释放算力可支持更复杂的灯效算法（如音乐频谱分析、端侧 AI 与灯效联动）。

灵活的刷新机制与中断管理：提供发送完成、半传输、字节完成、复位完成等多种中断，方便实现双缓冲切换、动态效果更新等高级功能。

第2章 使用方法

2.1 软件开发流程

1. 初始化 IO
2. 初始化 DMA
3. 初始化 ARGB
4. 使能 ARGB 发送使能 ARGB_Cmd(ENABLE);开始发送一帧数据
根据不同芯片，配置 ARGB 相应的 IO 为复用推挽模式，具体引脚功能可以参考官网中的数据手册，例如在 CH32V407 系列中，ARGB 的复用引脚是 PA15。CH32V407 配置代码如下：

```
void ARGB_GPIO_Init()
{
    GPIO_InitTypeDef GPIO_InitStructure;

    RCC_PB2PeriphClockCmd(RCC_PB2Periph_GPIOA, ENABLE);

    GPIO_InitStructure.GPIO_Pin   = GPIO_Pin_15;
    GPIO_InitStructure.GPIO_Mode  = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_High;
    GPIO_Init(GPIOA, &GPIO_InitStructure);
}
```

根据不同芯片，配置 DMA 通道，不同的芯片的 ARGB 可能有不同的与之绑定的 DMA 通道。DMA 的传输模式推荐固定为 DMA_Mode_Normal，DMA 的传输长度可按照需求配置，但是需要注意的是：DMA 的传输长度须严格与 ARGB 的数据刷新周期长度一致。CH32V407 配置代码如下：

```
void ARGB_WS2812B_DMA_Init(uint8_t *ARGB_DMA_Buffer, size_t buffer_size)
{
    RCC_HBPeriphClockCmd(RCC_HBPeriph_DMA1, ENABLE);

    DMA_InitTypeDef DMA_InitStructure;
    DMA_InitStructure.DMA_PeripheralBaseAddr = (uint32_t)&(ARGB->DATAR);
    DMA_InitStructure.DMA_MemoryBaseAddr    = (uint32_t)ARGB_DMA_Buffer;
    DMA_InitStructure.DMA_DIR                = DMA_DIR_PeripheralDST;
    DMA_InitStructure.DMA_BufferSize        = buffer_size;
    DMA_InitStructure.DMA_PeripheralInc     = DMA_PeripheralInc_Disable;
    DMA_InitStructure.DMA_MemoryInc        = DMA_MemoryInc_Enable;
    DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Byte;
    DMA_InitStructure.DMA_MemoryDataSize   = DMA_MemoryDataSize_Byte;
    DMA_InitStructure.DMA_Mode              = DMA_Mode_Normal;
    DMA_InitStructure.DMA_Priority          = DMA_Priority_High;
    DMA_InitStructure.DMA_M2M              = DMA_M2M_Disable;
    DMA_Init(DMA1_Channel10, &DMA_InitStructure);
    DMA_Cmd(DMA1_Channel10, ENABLE);
}
```

在配置 ARGB 时需要注意单线协议的参数如 T1H 是根据 ARGB 挂载的时钟总线的时钟来计算的，

例如在 CH32V407 系列中，ARGB 是在 HB 时钟总线下的，该系列的 HB 总线时钟是与内核时钟相同的，故可以直接使用 SystemCoreClock 来计算应写入寄存器的值，否则，根据系统时间结构或者直接调用 void RCC_GetClocksFreq(RCC_ClocksTypeDef *RCC_Clocks); 函数来获取当前平台下 ARGB 所在时钟总线的时钟，再去计算应写入寄存器的值。

```
#define ARGB_MS2HBTICK(ms) ((uint32_t)((float)SystemCoreClock / 1000.0f * (float)ms))
#define ARGB_US2HBTICK(us) ((uint32_t)((float)SystemCoreClock / 1000000.0f * (float)us))
#define ARGB_NS2HBTICK(ns) ((uint32_t)((float)SystemCoreClock / 1000000000.0f *
(float)ns))

void ARGB_WS2812B_Init(uint8_t *ARGB_DMA_Buffer, size_t buffer_size)
{
    // Enable the clock for ARGB peripheral
    RCC_HBPeriphClockCmd(RCC_HBPeriph_ARGB, ENABLE);

    // Initialize ARGB configuration structure
    ARGB_InitTypeDef ARGB_InitStruct;
    // Configure ARGB parameters
    ARGB_InitStruct.ARGB_Length      = buffer_size - 1;          // Set buffer length
(minus 1 for zero-based indexing)
    ARGB_InitStruct.ARGB_T1H         = ARGB_US2HBTICK(0.9f);    // Set high time for '1'
bit
    ARGB_InitStruct.ARGB_T0H         = ARGB_US2HBTICK(0.3f);    // Set high time for '0'
bit
    ARGB_InitStruct.ARGB_DataPeriod = ARGB_US2HBTICK(1.45f);    // Set data bit period
    ARGB_InitStruct.ARGB_RSTPeriod  = ARGB_US2HBTICK(80.0f);    // Set reset period
    ARGB_InitStruct.ARGB_Mode       = ARGB_Mode_SendRSTFirst;   // Set mode to send reset
first
    ARGB_InitStruct.ARGB_Endian     = ARGB_Endian_MSB;          // Set most significant
bit first
    ARGB_Init(&ARGB_InitStruct);                                // Apply ARGB configuration
    // Enable ARGB interrupt for frame reception complete
    ARGB_ITConfig(ARGB_IT_FRH, ENABLE);
    // Enable ARGB interrupt in the NVIC
    NVIC_EnableIRQ(ARGB_IRQn);
    ARGB_WS2812B_DMA_Init((void *)ARGB_DMA_Buffer, buffer_size);
}
```

需要注意的是：在重新开始一帧传输的时候，需要重新配置 DMA 的传输长度以及传输首地址。

示例

示例将在 8*8 像素数量由 WS2812 组成的点阵屏上显示从 GIF 文件生成的多帧 GRB 数据格式的位图数据。

```
int main(void)
{
    ...

    GD_ERR err;
    size_t err_pos;
    GD_GIF_HANDLE gif;
    gif = GD_FromMemory(media_default, media_default_end - media_default, &err,
&err_pos);
    GD_FRAME *frame = GD_GetFrame(gif, 0);
    ARGB_WS2812B_Init((uint8_t *)frame->Buffer, frame->Descriptor.Width * frame->
```

```
>Descriptor.Height * 3);  
  
    while (1)  
    {  
        for (uint32_t i = 0; i < GD_FrameCount(gif); i++)  
        {  
            GD_FRAME *frame = GD_GetFrame(gif, i);  
            ARGB_WS2812B_DMA_Init((uint8_t *)frame->Buffer, frame->Descriptor.Width *  
frame->Descriptor.Height * 3);  
            ARGB_Cmd(ENABLE);  
            while (DMA1_Channel10->CNTR != 0)  
            {  
            }  
            DMA_Cmd(DMA1_Channel10, DISABLE);  
            Delay_Ms(1000);  
        }  
    }  
}
```

2.2 典型应用场景

- 大规模智能照明系统
场景描述：城市景观亮化、建筑轮廓灯、大型舞台背景屏（数百米灯带，数万颗灯珠）。
方案特点：轻松驱动超长灯带不丢帧，支持打造同步且复杂的动态光效。
- 电竞外设与桌面氛围灯
场景描述：机械键盘背光、鼠标灯效、电脑机箱内部幻彩灯条、显示器偏光灯。
方案特点：支持四色 SK6812(RGBW)，色彩表现更细腻；硬件级低延迟，确保灯效能实时响应游戏画面和音乐节奏（如律动模式），无卡顿感。
- 汽车内饰与外部氛围灯
场景描述：车内脚窝灯、门板流水转向灯、格栅动态迎宾灯。
方案特点：高稳定性。ARGB 专用外设消除了软件模拟时序的不确定性，确保在车辆复杂电磁环境下灯光效果依然稳定可靠、无闪烁。
- 音乐可视化与互动装置
场景描述：蓝牙音箱律动灯、互动艺术装置、酒吧装饰灯。
方案特点：CPU 可全速运行 FFT（快速傅里叶变换）算法解析音频，同时将结果通过 DMA 发送给 ARGB 外设，光效与音乐同步简单。

历史版本

更新内容

日期	版本	变更内容
2026/03/26	V1.0	初版发行
2026/09/05	V1.1	优化组织措辞

声明

本手册仅供参考。作者在不另行通知的情况下，可随时进行修改。